Homework #1

(Due: Oct 16)

Task 1. [80 Points] Minimizing the Distance Between Two Pegged Frames



Figure 1: [Task 1] Lower the top frame until it touches the bottom frame.

You are given two $n \times n$ frames consisting of square cells, as shown in Figure 1, where n is a positive integer. The boundaries of both frames have negligible thickness and may be ignored. The height of each peg is also given. You may assume that no peg is shorter than 1 unit.

In the figure, the two frames are parallel, with the top frame directly above the bottom frame and their corresponding edges aligned. You may move the top frame horizontally in any direction, without changing its orientation, provided that some part of it, however small, remains vertically above some part of the bottom frame.

Your task is to determine the horizontal position of the top frame that allows it to be lowered as far as possible, that is, moved vertically downward without changing its orientation, before any part of it touches the bottom frame. But to make the the problem easier, we ask you to simply determine the smallest achievable distance of the form $\alpha^p + \alpha^q$ between the bases of the two frames after moving the top frame horizontally and then lowering it vertically, where $\alpha > 1$ is a given parameter and p and q are nonnegative integers.

Give an efficient algorithm for solving the problem above.

Task 2. [50 Points] Multiplying 3×3 Matrices Using Fewer Than 23 Products?

Based on what we learned from the Lecture (Lecture 3) on Strassen's algorithm, we can say that two 2×2 matrices consisting of real-valued entries can be multiplied using only 7 scalar products. But how about multiplying two such 3×3 matrices? The claim below is that they can be multiplied using only 21 scalar products (compare that with the corresponding claim on the last slide of Lecture 3).

Let

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}, \qquad B = \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{pmatrix},$$

and let

$$C = AB = \begin{pmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{pmatrix}.$$

Assuming that the scalar entries commute, the product can be computed using only 21 scalar multiplications.

First compute the following three products.

$$\begin{aligned} q_1 &= b_{12}b_{21} \\ q_2 &= b_{13}b_{31} \\ q_3 &= b_{23}b_{32} \end{aligned}$$

Next compute the following eighteen products.

$$\begin{aligned} p_{11} &= (a_{12} + b_{12})(a_{11} + b_{21}) \\ p_{12} &= (a_{13} + b_{13})(a_{11} + b_{31}) \\ p_{13} &= (a_{13} + b_{23})(a_{12} + b_{32}) \\ p_{14} &= a_{11}(b_{11} - b_{12} - b_{13} - a_{12} - a_{13}) \\ p_{15} &= a_{12}(b_{22} - b_{21} - b_{23} - a_{11} - a_{13}) \\ p_{16} &= a_{13}(b_{33} - b_{31} - b_{32} - a_{11} - a_{12}) \end{aligned}$$

$$\begin{aligned} p_{21} &= (a_{22} + b_{12})(a_{21} + b_{21}) \\ p_{22} &= (a_{23} + b_{13})(a_{21} + b_{31}) \\ p_{23} &= (a_{23} + b_{23})(a_{22} + b_{32}) \\ p_{24} &= a_{21}(b_{11} - b_{12} - b_{13} - a_{22} - a_{23}) \\ p_{25} &= a_{22}(b_{22} - b_{21} - b_{23} - a_{21} - a_{23}) \\ p_{26} &= a_{23}(b_{33} - b_{31} - b_{32} - a_{21} - a_{22}) \end{aligned}$$

$$
\begin{aligned}
p_{31} &= (a_{32} + b_{12})(a_{31} + b_{21}) \\
p_{32} &= (a_{33} + b_{13})(a_{31} + b_{31}) \\
p_{33} &= (a_{33} + b_{23})(a_{32} + b_{32}) \\
p_{34} &= a_{31}(b_{11} - b_{12} - b_{13} - a_{32} - a_{33}) \\
p_{35} &= a_{32}(b_{22} - b_{21} - b_{23} - a_{31} - a_{33}) \\
p_{36} &= a_{33}(b_{33} - b_{31} - b_{32} - a_{31} - a_{32})
\end{aligned}
$$

The entries of C are then recovered using only additions and subtractions.

$$
\begin{aligned}
c_{11} &= p_{14} + p_{11} + p_{12} - q_1 - q_2 \\
c_{12} &= p_{15} + p_{11} + p_{13} - q_1 - q_3 \\
c_{13} &= p_{16} + p_{12} + p_{13} - q_2 - q_3 \\
\\
c_{21} &= p_{24} + p_{21} + p_{22} - q_1 - q_2 \\
c_{22} &= p_{25} + p_{21} + p_{23} - q_1 - q_3 \\
c_{23} &= p_{26} + p_{22} + p_{23} - q_2 - q_3 \\
\\
c_{31} &= p_{34} + p_{31} + p_{32} - q_1 - q_2 \\
c_{32} &= p_{35} + p_{31} + p_{33} - q_1 - q_3 \\
c_{33} &= p_{36} + p_{32} + p_{33} - q_2 - q_3
\end{aligned}
$$

Include sufficient explanation in each of your answers below.

(a) [**20 Points**] Can you find a counter-example establishing that the algorithm above is incorrect?

(b) [**30 Points**] If your answer to part (a) was affirmative, explain why the algorithm is incorrect. If not, can you use it to design a recursive divide-and-conquer algorithm for multiplying two $n \times n$ matrices in $o\left(n^{\log_2 7}\right)$ time, where $n = 3^k$ for some integer $k \geq 0$ (the way did it for Strassen's algorithm)?

Task 3. [50 Points] Running Strassen's Algorithm Under Memory Constraints

Recall that within each recursive call, Strassen's algorithm allocates extra memory (space) for temporary matrices and deallocates this memory at the end of the call. A recursive call requires a minimum amount of extra memory to proceed. This extra space enables Strassen's algorithm to perform asymptotically less work than the standard cubic-time matrix multiplication algorithm.

(a) [**15 Points**] What is the minimum amount of extra space required for Strassen's algorithm to multiply two $n \times n$ matrices?

(*b*) [**35 Points**] Suppose you are running Strassen's algorithm in an environment where the amount of available memory fluctuates over time. As a result, some recursive calls may be unable to allocate the extra memory they require, although additional memory may become available later. Modify the algorithm so that recursive calls can continue making progress even when the required extra memory is unavailable, while still taking advantage of additional memory if it becomes available later. Blocking a recursive call until sufficient memory becomes available is not allowed. Give pseudocode for your modified algorithm.

Task 4. [70 Points] A New Transform

```
ForwardTransform( u, i_u, v, i_v, n )                    {n = 2^k for k ∈ ℤ}
 1. if n = 1 then v[i_v] ← u[i_u]
 2. else
 3.     m ← n/2,  r ← 1
 4.     while n > 2 do
 5.         n ← n/2,  r ← 3 × r
 6.     endwhile
 7.     ForwardTransform( u, i_u + m, v, i_v + r + r, m )
 8.     for t ← 0 to m − 1 do
 9.         v[i_v + r + t] ← u[i_u + t]
10.         u[i_u + t] ← u[i_u + t] + u[i_u + m + t]
11.     endfor
12.     ForwardTransform( u, i_u, v, i_v, m )
13.     for t ← 0 to m − 1 do
14.         u[i_u + t] ← v[i_v + r + t]
15.     endfor
16.     ForwardTransform( u, i_u, v, i_v + r, m )
17. endif
```

Figure 2: [Task 4] The function transforms a vector $u[i_u : i_u + n - 1]$ to a vector $v[i_v : i_v + n_v - 1]$, where $n = 2^k$ for $k \in \mathbb{Z}$, and n_v is the length of the transformed vector.

The ForwardTransform transforms a given vector $u[i_u : i_u + n - 1]$ to a vector $v[i_v : i_v + n_v - 1]$, where $n = 2^k$ for $k \in \mathbb{Z}$, and n_v is the length of the transformed vector.

(*a*) [**10 Points**] What is the length of the transformed vector as a function of n? What is the running time of ForwardTransform?

(*b*) [**35 Points**] Design a BackwardTransform function that given the transformed vector v returns the original vector u. What is its running time?

(*c*) [**25 Points**] Explain how you would use the forward and backward transforms above to multiply two polynomials of degree bound n each given in the coefficient form in $o\left(n^2\right)$ time.